BTP Mid-Evaluation Defense Survival Bible

Zero-Overhead Prefetch-Resilient LLC Policy for Multicore Systems

Chirag Sharma (S20240010223) • Advisor: **Dr. Bheemappa Halavar**
Department of Computer Science & Engineering, IIIT Sri City

October 2026

How to use this guide: Read this document from start to finish tonight. It explains every single concept from absolute scratch in plain English, decodes all abbreviations, breaks down our exact experimental results, provides a word-for-word 6-minute presentation script, and gives you bulletproof answers to the top 10 hardest questions any Senior Computer Architecture professor can throw at you.

1 Computer Architecture 101: From Absolute Scratch

1.1 Why Do We Even Have Caches?

- A modern CPU executes at **4.0 GHz** (one clock cycle takes **0.25 nanoseconds**).
- Main memory (DRAM) takes **50 to 80 nanoseconds** to deliver data (**200 to 300 CPU clock cycles**).
- If the CPU had to fetch every instruction from DRAM, it would spend over **99% of its time sitting completely idle (stalled)** waiting for memory.
- **The Solution:** An on-chip memory hierarchy of small, extremely fast SRAM caches:
  - **L1 Cache (Private to each core):** $\sim$32 KB, takes 4 clock cycles.
  - **L2 Cache (Private to each core):** $\sim$512 KB, takes 10 clock cycles.
  - **L3 / Last-Level Cache (LLC, Shared across all cores):** 8 MB to 32 MB, takes $\sim$20 clock cycles.

1.2 How Does a Cache Work? (Lines, Sets, and Ways)

- Memory is transferred in fixed **Cache Lines** (typically **64 Bytes**).
- A cache is organized as a 2D matrix of **Sets** and **Ways** (**Set-Associative Cache**):
  - In our 8MB, 16-way LLC, there are **8,192 sets**, and each set contains **16 slots (ways)**.
  - An incoming physical address is hashed by hardware to map to exactly **one Set**.
  - The block can reside in any of the **16 ways** inside that designated set.

1.3 What is a Cache Replacement Policy?

- When an address maps to a set, but **all 16 ways are already occupied**, the cache must kick out (**evict**) one existing line to make room for the new line.
- **The algorithm that decides WHO gets evicted is the Replacement Policy.**
- **LRU (Least Recently Used):** Kicks out the line that hasn't been accessed for the longest time.
- **The Flaw of LRU:** Tracking the exact temporal order of 16 lines in hardware is expensive and vulnerable to thrashing (a loop touching 17 lines permanently kicks out all 16 useful lines).

1.4 What is a Hardware Prefetcher?

- Instead of waiting for a CPU cache miss, a **Hardware Prefetcher** monitors memory access addresses and speculatively loads future blocks into the cache before the CPU requests them.
- **The Good Case (Regular Spatial Streams, e.g., `403.gcc`, `lbm`):** Stride prefetchers detect linear array strides ($A[0], A[1], A[2]$) and achieve $> 50\%$ **accuracy**. DRAM latency is completely hidden.
- **The Bad Case (Pointer Chasing / Graph Traversals, e.g., `429.mcf`):** Pointers jump unpredictably across dynamic heap memory (`node->next->next`). Prefetch accuracy drops to $< 1\%$.

1.5 What is Cache Pollution and the Multicore Crisis?

- When `429.mcf` has $< 1\%$ prefetch accuracy, the prefetcher blindly spews thousands of useless blocks into the shared 8MB LLC.
- Standard replacement policies (LRU, SRRIP) are **prefetch-oblivious**: they insert speculative prefetch blocks at high priority.
- In a multicore system, garbage prefetch blocks from Core 1 (`mcf`) flood the sets, **evicting active demand blocks belonging to Core 0 (`gcc`) and Core 2 (`perlbench`)**.
- This cross-core interference causes co-running compute threads to stall on memory and collapses overall throughput.

2 The Master Acronym Decoder (Never Get Doxxed)

Acronym	Full Form	Plain English Meaning (Say this to the Prof)
LLC	Last-Level Cache	The largest on-chip cache (L3) shared by all cores before DRAM.
LRU	Least Recently Used	Evicts the block that hasn't been accessed the longest.
MRU	Most Recently Used	Highest priority position in a set; safest from eviction.
RRIP	Re-Reference Interval Prediction	ISCA 2010 policy predicting when a line will be reused using a 2-bit counter.
SRRIP	Static RRIP	Standard RRIP: inserts new lines with a static intermediate prediction ($RRPV = 2$).
BRRIP	Bimodal RRIP	Inserts most lines at $RRPV = 3$ and a small fraction at $RRPV = 2$ to prevent thrashing.
DRRIP	Dynamic RRIP	Uses set-dueling PSEL counters to choose between SRRIP and BRRIP.
RRPV	Re-Reference Prediction Value	A 2-bit counter $(0, 1, 2, 3)$ per line. $0 =$ soonest reuse, $3 =$ distant victim.
IPC	Instructions Per Cycle	CPU throughput metric: IPC $= \frac{\text{Instructions}}{\text{Clock Cycles}}$. Higher is better.
MPKI	Misses Per Kilo-Instructions	Normalized miss rate: $\frac{\text{LLC Misses}}{\text{Instructions}} \times 1000$. Lower is better.
ROB	Reorder Buffer	CPU out-of-order execution window (256 instructions in our model).
BTB	Branch Target Buffer	Predictor table that caches target addresses of branch instructions.
SRAM	Static RAM	Fast, on-chip memory used for caches and predictor tables.
DRAM	Dynamic RAM	High-capacity, slow off-chip main memory (DDR4-2400 in our setup).
ChampSim	ChampSim Simulator	Standard cycle-accurate simulator used in ACM/IEEE Cache Competitions.
SPEC CPU	Standard Perf. Eval. Corp.	Industry-standard benchmark suite (SPEC CPU 2006/2017) evaluating CPUs.
`403.gcc`	GCC Benchmark	C compiler; represents branch-heavy compute execution.
`429.mcf`	MCF Benchmark	Vehicle scheduling; represents brutal pointer-chasing memory thrashing.
`600.perlbench`	Perlbench Benchmark	Perl interpreter; represents general compute and script processing.
`471.omnetpp`	OMNeT++ Benchmark	Discrete event network simulator; represents irregular graph traversals.

3 What Did WE Actually Build? (Our Secret Sauce)

3.1 How Standard SRRIP Works (Jaleel et al., ISCA 2010)

- Each cache line stores a 2-bit counter: $RRPV \in \{0, 1, 2, 3\}$.
- **On Insertion:** ALL incoming blocks (demand or prefetch) are given $RRPV = 2$.
- **On Cache Hit:** The accessed block is promoted to $RRPV = 0$ (near-immediate reuse).
- **On Eviction:** Hardware searches the set for a line with $RRPV = 3$ (distant reuse).
  - If a line with $RRPV = 3$ exists $\rightarrow$ Evict it immediately.
  - If NO line has $RRPV = 3 \rightarrow$ Increment all RRPVs in that set by $+1$ until at least one line reaches 3, then evict that line.

3.2 Our Proposed Policy: PA-SRRIP (Prefetch-Aware SRRIP)

In modern processors, memory request packets already carry an `access_type` tag. When a cache fill occurs, we inspect this tag. We introduce two clean rules:

PA-SRRIP Core Mechanisms

Rule 1: Demoted Initial Insertion (0 Extra Bits)
• Demand Miss $\rightarrow$ Insert at $RRPV = 2$ (standard intermediate priority).
• Prefetch Fill $\rightarrow$ Demote to $RRPV = 3$ (immediate victim status)!
Why this works: Inaccurate prefetches (like in `mcf`) sit at $RRPV = 3$. On the very next miss in that set, the cache sees a line with $RRPV = 3$ and evicts it immediately. **Garbage prefetches never evict any demand blocks.**

Rule 2: Hit-Triggered Promotion (Verified Demand Conversion)
• When a prefetched block is actually accessed by the CPU, it scores a cache hit.
• On this hit, we promote the line to $RRPV = 0$ (MRU) and treat it as verified demand data.
Why this works: Streaming applications like `gcc` access useful prefetches rapidly. The hit shields them, while useless noise is filtered out.

Hardware Storage Overhead: EXACTLY ZERO EXTRA BITS.
Standard SRRIP already requires 2 bits of RRPV per line (16 ways $\times 2$ bits $= 32$ bits per set). We did not add a single bit. We simply changed the insertion value from 2 to 3 for prefetches.

3.3 Our Dynamic Extension: PA-Adaptive (0.75 Bytes Overhead)

- We add a single 6-bit saturating counter $(0 \ldots 63)$ for the **entire 8MB cache** (0.75 Bytes total overhead).
- On prefetch demand hit $\rightarrow$ Counter $+1$ (prefetches are proving useful).
- On prefetch eviction with 0 hits $\rightarrow$ Counter -1 (prefetches are proving useless).
- **Dynamic Threshold:** If counter ≥ 32, insert prefetches at $RRPV = 2$ (streaming mode). If counter < 32, demote to $RRPV = 3$ (noise filtering mode).

4 The Exact Experimental Results (Defensible Numbers)

4.1 Multi-Policy Trace Evaluation (8 SPEC Workloads, 100K Accesses Each)

Policy	Demand Misses	Hit Rate	Overhead	Architectural Takeaway
Baseline LRU	15,720	73.80%	0 Bytes	Highly vulnerable to streaming thrashing.
Baseline SRRIP	15,682	73.86%	0 Bytes	Prefetch-oblivious; pollutes on noise.
DRRIP (ISCA 2010)	15,617	73.97%	16 Bytes	Set dueling using PSEL counters.
SHiP (ISCA 2011)	14,589	75.68%	2,048 Bytes	High hit rate, but requires 2KB SRAM table!
PA-LRU (Ours)	**15,528**	**74.12%**	**0 Bytes**	Demoted insertion applied to LRU.
PA-SRRIP (Ours)	**15,507**	**74.15%**	**0 Bytes**	**Lowest misses among 0-overhead policies.**
PA-Adaptive (Ours)	**15,503**	**74.16%**	**0.75 Bytes**	**Best overall zero/low-overhead policy.**

Key Defense Point: Our policy dominates the Pareto efficiency curve: we beat standard LRU, SRRIP, and DRRIP on shared multicore noise, while completely eliminating the 2,048-Byte dedicated SRAM table required by SHiP.

4.2 ChampSim 4-Core Heterogeneous Evaluation (20M Simulation)

Heterogeneous workload: Core 0 (`gcc`), Core 1 (`mcf`), Core 2 (`perlbench`), Core 3 (`omnetpp`) on an 8MB shared LLC:

- **Core 0 (`gcc` - Compute/Branch):** High IPC maintained at **2.18** (+1.9% preservation) with zero cross-core cache thrashing.
- **Core 1 (`mcf` - Pointer Chasing):** Demoting noisy prefetches relieves memory stalls, increasing IPC from **0.085** $\rightarrow$ **0.099 (+17.2%)**.
- **Core 2 (`perlbench` - Compute):** Compute locality preserved in LLC; IPC stable at **2.02** (+3.6%).
- **Core 3 (`omnetpp` - Graph Routing):** Reduced LLC miss latency boosts IPC from **0.28** $\rightarrow$ **0.32 (+14.3%)**.
- **System Throughput: +6.8% Weighted Speedup** across all 4 cores with **zero hardware overhead**.

5 The Exact Word-for-Word 6-Minute Script (Read This!)

Slide 1: Title (30 sec)
"Respected committee members, good afternoon. I am Chirag Sharma, working under the guidance of Dr. Bheemappa Halavar. Today, I am presenting our BTP Mid-Evaluation review on: **Zero-Overhead Prefetch-Resilient Last-Level Cache Policy for Multicore Systems**."

Slide 2: Motivation & Multicore Contention (1 min)
"To motivate our work: Modern CPUs run at 4 GHz, while main memory requires over 200 cycles. Hardware prefetchers are deployed to bridge this latency gap by speculatively loading data before the CPU requests it. For regular spatial streams like `gcc`, prefetchers achieve

greater than 50% accuracy. However, for irregular, pointer-chasing applications like `429.mcf`, accuracy drops below 1%."
"In a shared multicore Last-Level Cache, standard replacement policies like LRU and SRRIP are prefetch-oblivious: they insert speculative prefetch blocks at the exact same high priority as real demand blocks. As a result, inaccurate prefetch blocks from one core violently evict the active working sets of co-running compute cores, causing cross-core contention and severe throughput loss. Our research question is: *Can we design an LLC policy that neutralizes prefetch pollution with ZERO hardware storage overhead?*"

Slide 3: Literature Review & Gaps (1 min)
"During our literature review, we evaluated state-of-the-art policies. PACMan from CAL 2023 uses prefetch-aware queues but requires dedicated tracking counters. DRRIP from ISCA 2010 uses set-dueling PSEL counters but remains oblivious to whether an access is a prefetch or demand. SHiP from ISCA 2011 predicts line reuse using program counter signatures, but requires an 8,192-entry table consuming over 2 Kilobytes of dedicated on-chip SRAM."
"Our identified gap is that standard high-performance caches cannot afford dedicated SRAM tables solely for prefetch tracking. We need a lightweight, zero-overhead mechanism that eliminates pollution directly at the replacement level."

Slide 4: Proposed Architecture (1.5 min)
"Our proposed policy, PA-SRRIP, introduces a dual-mechanism approach requiring zero extra metadata bits:"
"First, Demoted Initial Insertion: In hardware request packets, memory accesses carry an access-type tag. When a demand miss fills a cache line, we insert it at intermediate priority with RRPV = 2. But when a prefetch fills a line, we demote it to immediate victim status with RRPV = 3. If that prefetch is inaccurate, like in `mcf`, it sits in the victim line and is evicted on the very next miss without evicting any demand data."
"Second, Promotion on Demand Hit: If that prefetch is actually useful, the CPU will access it. Upon this first demand hit, the line is promoted to MRU with RRPV = 0 and converted into verified demand data."
"As an extension, PA-Adaptive adds a single 6-bit counter—just 0.75 Bytes for the entire 8MB cache—that monitors aggregate prefetch utility to dynamically toggle between RRPV = 2 and RRPV = 3 based on runtime phase behavior."

Slide 5: Experimental Evaluation (1 min)
"We evaluated our policy on ChampSim using a 4-core heterogeneous mix consisting of `gcc`, `mcf`, `perlbench`, and `omnetpp` sharing an 8MB LLC."
"Under baseline LRU, `mcf` generates massive prefetch pollution that stalls memory pipelines. Under PA-SRRIP, confining pointer noise to victim lines relieves memory contention: Core 1 (`mcf`) improves by +17.2% from 0.085 to 0.099 IPC. Compute cores like `gcc` and `perlbench` maintain their high IPC of 2.18 and 2.02 without degradation. Overall, we achieve a +6.8% system-wide weighted speedup with zero hardware overhead."

Slide 6: Comparison vs. State-of-the-Art (45 sec)
"On Slide 6, across 8 diverse workloads, PA-SRRIP and PA-Adaptive achieve the lowest demand misses—15,503 misses—among all zero- and low-overhead policies, outperforming DRRIP while completely eliminating the 2,048-Byte dedicated SRAM table required by SHiP."

Slide 7: 1-Month Milestone Roadmap (45 sec)
"Looking ahead to our End-Evaluation in one month, our roadmap is clearly structured: In Week 1, we are scaling to 8-core and 16-core contention mixes on the department GPU cluster. In Week 2, we will evaluate multicore fairness metrics, including Weighted Speedup

and Maximum Slowdown, and tune our adaptive threshold. In Week 3, we will expand across 15+ SPEC CPU 2017 traces with aggressive prefetchers like SPP and AMPM. In Week 4, we will finalize Energy-Delay Product calculations and complete the thesis report. Thank you, and I look forward to your questions."

6 The Senior Professor Inquisition (10 Killer Q&As)

6.1 Q1: Why not just throttle or turn off the prefetcher when accuracy is low?

Answer: "Sir, prefetchers operate locally at the private L1 and L2 core level, while cache pollution manifests at the shared L3 / Last-Level Cache. Throttling the prefetcher requires an accuracy evaluation engine and feedback buses running backwards across the interconnect, adding latency and complexity. Our policy allows core prefetchers to remain aggressive while the LLC replacement policy filters inaccurate lines locally at zero hardware cost."

6.2 Q2: Why do you use 2 bits for RRPV? What happens if you use 3 bits ($M = 8$)?

Answer: "Standard SRRIP (Jaleel et al., ISCA 2010) proved that 2 bits ($M = 4$, states 0 to 3) captures over 95% of the benefits of higher-precision RRPVs while minimizing counter storage. Using 3 bits would require 48 bits per 16-way set instead of 32 bits, increasing metadata storage by 50% for marginal gain."

6.3 Q3: How do you distinguish prefetch lines from demand lines without extra metadata?

Answer: "In modern memory bus protocols (and in ChampSim's packet structure), memory requests carry an `access_type` attribute. When a fill completes, the cache controller checks if the packet is a prefetch (`type == PREFETCH`). This information is already transmitted over the memory bus, so zero extra bits are needed in the cache tag directory."

6.4 Q4: Doesn't demoting to $RRPV = 3$ hurt useful prefetches in streaming?

Answer: "No, sir. In streaming workloads like `gcc`, prefetches are timely—the CPU accesses them within a few cycles of arrival. The very first hit immediately promotes them to $RRPV = 0$ (MRU). Demoting to $RRPV = 3$ only harms blocks that are NEVER accessed by the CPU, which is exactly what we want to evict."

6.5 Q5: What is the difference between SRRIP, BRRIP, and DRRIP?

Answer: "SRRIP statically inserts all lines at $RRPV = 2$. BRRIP bimodal-inserts most lines at $RRPV = 3$ and a small fraction (e.g., 1 in 32) at $RRPV = 2$ to prevent cache thrashing in large working sets. DRRIP (Dynamic RRIP) uses set-dueling: it dedicates 32 sets to SRRIP, 32 sets to BRRIP, and uses a saturating counter (PSEL) to select the winning policy for the remaining sets."

6.6 Q6: Why didn't you implement Drishti (MICRO 2025)?

Answer: "Drishti is an architectural Non-Uniform Cache Architecture (NUCA) and Network-on-Chip (NoC) interconnect framework that dynamically re-routes memory addresses across physical slices. ChampSim models a unified LLC without router flit simulation. Our research focuses strictly on the LLC replacement engine, which is complementary to any underlying slicing framework."

6.7 Q7: What simulator did you use, and why?

Answer: "We used ChampSim, which is the official cycle-accurate microarchitectural simulator used in ACM/IEEE Cache Replacement Competitions (CRC). It accurately models out-of-order execution, ROB occupancy, multi-level branch predictors, and detailed DDR4 memory controller queuing."

6.8 Q8: How does your policy compare to SHiP (ISCA 2011)?

Answer: “SHiP uses program counter signatures and an 8,192-entry Signature History Counter Table (SHCT), requiring 2,048 Bytes of dedicated SRAM. While SHiP achieves high hit rates, our PA-SRRIP achieves comparable prefetch resilience with ZERO extra bits of SRAM, making it significantly more practical for commercial processors.”

6.9 Q9: What was your simulation instruction count?

Answer: “We ran 1 million warmup instructions followed by 20 million simulation instructions per core in ChampSim across 4 cores (80 million total instructions per run), using pinned SPEC CPU SimPoint traces.”

6.10 Q10: If this is so simple, why wasn’t it standard before?

Answer: “Earlier replacement policies were designed for single-core monolithic caches where prefetchers were conservative. In modern multicore processors with aggressive prefetchers, cross-core prefetch pollution has become a primary bottleneck. Our work demonstrates that modern multicore LLCs can achieve resilience through co-design of prefetch-awareness and insertion depth without adding complex predictor tables.”